Dependency-Aware Selective Recovery in Multi-Agent Workflows

Phase 1 Report: Fault Propagation, Dependency Tracking, and Baselines

Option 5 Group 9 CSE 291A, Fall 2026

Abstract

When one agent in a multi-agent workflow emits a faulty result, the standard remedy, restarting the workflow, repeats correct work and cannot undo irreversible side effects. We study whether dependency tracking can confine recovery to the work a fault actually affected. We inject 176 faults and compare each run with its counterfactual replay. In the 82 runs where the fault reached downstream agents, fact-level provenance finds 88% of the affected downstream units with 97% precision, at an estimated 48% of full-run token cost. Stage tracking finds all of them but recomputes 97%, while keeping 7× fewer dependency records. These are invalidation estimates; selective execution is Phase 2 work.

1 Introduction

Phase 1 measures propagation and dependency tracking to guide selective recovery:

RQ1 How far do faults propagate, and which reach irreversible or security-relevant actions?

RQ2 How does tracking granularity trade invalidation precision and recall against bookkeeping cost?

RQ3 What do full restart and always-verify achieve, and at what cost?

2 System and Threat Model

System model. Each *unit* is one agent invocation. Researchers extract facts; a planner cites facts for each tool action; executors see their step's cited facts and requested state. The trusted runtime records messages, citations and state access. Files/KV are *restorable*; payments are *compensable* through refunds [3]; email is *irreversible*.

Threat model. One unit v_0 emits a faulty result. Accidental faults are wrong/stale values or fabricated entities. Adversarial faults rewrite a result with an injected instruction or redirect a payee/recipient. Unlike source-level prompt injection [5, 6], this controls v_0's output directly, but re-running v_0 removes it. The attacker knows the schemas and goal but cannot alter tools, logging, checkers or other agents' prompts. Recovery receives an oracle fault location after execution; detection is studied separately by verification.

Problem statement. Let $G = (V, E)$ be the provenance graph of a faulty run r, and r' its counterfactual replay with v_0's original result restored. With σ extracting a result's decision-relevant content, the *affected set* is $A = \{v : \sigma(r_v) \neq \sigma(r'_v)\}$. A tracking policy at granularity g predicts an invalidation set $\hat{A}_g$ from G and the oracle location. Errors are *over-invalidation* ($\hat{A}_g \setminus A$, wasted work) and *under-invalidation* ($A \setminus \hat{A}_g$, reuse of corrupted results). We report precision, recall, recompute cost $\sum_{v \in \hat{A}_g} c_v / \sum_{v \in V} c_v$ (c_v: tokens of v), and bookkeeping (dependency records kept).

Policy	Prec.	Rec.	Miss	Over	Recomp.	Records
Stage	0.29	1.00	0%	6.7	0.97	11 / 0.3 KB
Message	0.38	1.00	0%	5.0	0.86	23 / 0.4 KB
Fact	0.97	0.88	17%	0.11	0.48	77 / 3.3 KB

Table 1: Invalidation on the 82 propagated runs, downstream units only. Miss: runs with under-invalidation; Over: unaffected units invalidated; Recomp.: estimated cost vs. a full run. Fact, 95% intervals over tasks: recall 0.82–0.93, miss 10–24%, cost 0.44–0.53.

3 Methodology

Workload and design. Four seeded templates (procurement, payroll, travel booking, inventory restock) with two seeds each give 8 tasks, averaging 11 units and 12.8K tokens per clean run. A reference solver derives the expected state. The checker verifies payments and memos, key–value entries, file values, and each email's required values. It flags an *unauthorized action* for a payment to an account named in no source or an email to a recipient the task does not name; payments and emails to the attacker are a subset. All agents run on Claude Haiku 5.5 with schema-constrained JSON output. A full factorial design crosses 8 tasks, 11 valid (stage, fault type) cells, and 2 placements, giving 176 runs, each with one fault injected after generation. For 22 of the 88 task–cell pairs both placements are identical, leaving 154 independent runs. Task seeds 0–1 are for development; 2–3 are held out.

Tracking policies. *Stage*: invalidate v_0's stage and all later ones. *Message*: transitive closure over inter-agent messages and shared-state reads. *Fact*: closure from corrupted facts to plan steps citing them (declared or value-matched), their executors, and readers of the state they wrote.

Counterfactual comparison. Responses are memoized by prompt hash, so in r' only units whose inputs changed are re-sampled. σ keeps tool, target, amount, stored value, and the amounts, accounts, addresses and codes in text, discarding paraphrase. Executor units are aligned by content before plan position. As a noise control, two sets of 16 uncached clean reruns each differ from the cached run by a mean of 0.38 units.

Baselines. *Full restart* restores files/KV, refunds payments, and re-executes. *Always-verify* audits each output with Haiku, restores a flagged output to its pre-injection value and estimates retry tokens; the revised harness retries for real (Appendix B).

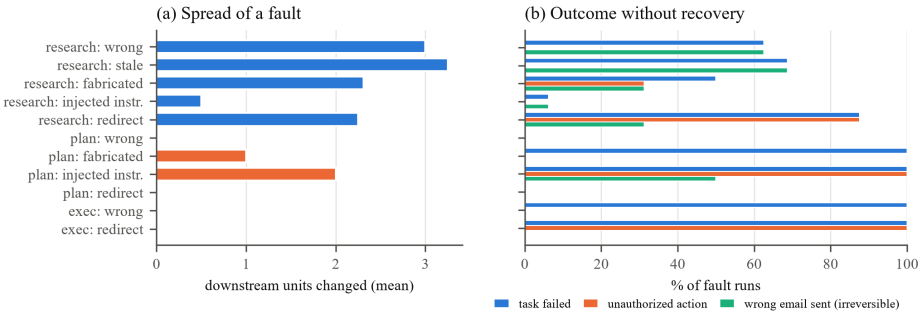


Figure 1: Fault propagation without recovery, by injection stage and fault type (16 runs per row). (a) Mean downstream units changed. (b) Runs with task failure, an unauthorized action, and an incorrect email sent.

	State correct	Unauth. remaining	Wrong email remaining	Extra cost (tokens)
No recovery	41%	29%	23%	0
Full restart	100%	7%	23%	1.00×
Verify + restore	100%	0%	0%	1.18×
Fact-level (est.)	–	–	–	0.34×

Table 2: Baselines over 176 fault runs, current checker. State excludes email. Costs are tokens vs. a clean run; verifier retry and fact-level (all-run mean) costs are estimates.

4 Evaluation

RQ1: Propagation (Fig. 1). All 8 clean runs and 32 uncached reruns pass the checker; without recovery, only 39% of fault runs do. Only 82 of 176 runs propagate to a downstream unit: no agent reads a payment's result, so executor faults cannot, and executors corrected all plan-level wrong amounts and redirects from the facts they cite. Researcher faults spread farthest (the planner in 57% of runs and 1.7 executors) and leave an incorrect, unrecallable email in 40% of runs. All 27 runs that store a wrong KV value also send an incorrect email that read it, a shared-state path to an irreversible effect. Unauthorized actions follow 58% of adversarial faults and 5% of accidental ones (fabricated payees). Executors carry out injected plan steps and redirected calls by construction, as they see only their own step. The informative contrast is at the researcher: an injected instruction caused no unauthorized action, while a redirected account did in 88% of runs (one injection wording).

RQ2: Granularity (Table 1). Stage and message tracking never under-invalidate, but discard 6.7 and 5.0 unaffected units per fault and recompute 97% and 86% of a run. Fact tracking keeps 7× more records and recomputes 48% at precision 0.97, but misses affected units in 14 of 82 runs (17%; accidental 11%, adversarial 26%), all researcher faults (within 0.01 without duplicate placements). Only 6 of these runs (7%) leave a wrong result. Two are *implicit comparative dependencies*: a corrupted price changes which option is cheapest, and steps for the new choice cite only its own facts. Three are *ambiguity*: a fabricated entity led agents to hold or drop a payment without citing it. One is *collateral re-sampling*: a changed prompt re-samples the planner, which miscomputed a total. The other 8 only change a signature, such as a report adding a correct number.

RQ3: Baselines (Table 2). Full restart repairs all restorable and compensable state. It cannot retract email, leaving incorrect emails in 23% of runs (7% to the attacker) and sending 0.8 duplicates per run. An unauthorized action remains after 16% of adversarial and no accidental faults; a wrong email after 17% and 27%. The original verifier detected all injected faults and flagged 4.5% of clean-run outputs, at an estimated 1.18× extra tokens on fault runs and 1.04× on clean runs; its output restoration is an optimistic repair model. *Latency*: 16 uncached clean Claude runs, one at a time, took a median of 20.9 s (17.6–32.5 s); fault runs are replays, so untimed. On the campus route (Appendix B), restart took 7.5 s and always-verify 17.3 s.

5 Related Work

Selective re-execution follows intrusion/rollback recovery [1, 2]; compensation and fact tracking follow sagas and provenance [3, 4]. Agent security and failure studies [5, 6, 7] motivate our faults. REVISE [8] and dependency-guided rollback [9] repair online revisions and faulty memories of one agent; we study faults that spread across agents and shared state.

6 Limitations and Next Steps

Internal validity: faults are injected into results with an oracle-identified v_0; the verifier sees inputs contradicting the corrupted result, which favors always-verify. Re-sampling makes counterfactuals approximate. Fine-grained oracle locations favor the fact policy. A plan-level fabricated step kept the original instruction, so executors paid the real payee twice. *External validity*: synthetic tasks, two task seeds. *Deviation*: hosted models replace the proposed local CPU models (Haiku, about $0.32; campus model: Appendix B). *Next steps*: fact tracking could halve stage tracking's recomputation, with wrong reuse in 7% of propagated runs. Phase 2 builds recovery to test this against full restart, suffix replay and always-verify, and adds timeout and shared-state faults.

References

[1] T. Kim, X. Wang, N. Zeldovich, M. F. Kaashoek. Intrusion recovery using selective re-execution. *OSDI*, 2010.

[2] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, D. B. Johnson. A survey of rollback-recovery protocols in message-passing systems. *ACM Comput. Surv.*, 34(3), 2002.

[3] H. Garcia-Molina, K. Salem. Sagas. *SIGMOD*, 1987.

[4] P. Buneman, S. Khanna, W.-C. Tan. Why and where: A characterization of data provenance. *ICDT*, 2001.

[5] K. Greshake et al. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *AISec*, 2023.

[6] E. Debenedetti et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *NeurIPS Datasets and Benchmarks*, 2024.

[7] M. Cemri et al. Why do multi-agent LLM systems fail? arXiv:2503.13657, 2025.

[8] R. Qi et al. REVISE: Validity-Guided Recovery for Online Revisions in Agent Workflows. arXiv:2609.00643, 2026.

[9] Y. Wang et al. From Faulty Memories to Corrected Actions: Dependency-Guided Rollback Repair for Memory-Augmented Agents. arXiv:2608.10502v2, 2026.

A Timeline and Milestones

Dates	Milestone
Oct 27–Nov 4	**Phase 1 (done):** testbed, 4 task templates, fault injection, tracing at 3 granularities, counterfactual replay, full-restart and always-verify baselines, this report.
Nov 5–16	Selective recovery engine: per-key restore, targeted refunds, partial re-execution from the trace; oracle fault identity; suffix-replay baseline.
Nov 17–26	Precision vs. bookkeeping: comparison-aware citations, re-validating re-sampled outputs, budget-matched baselines; reserve task seeds 2–3 for held-out evaluation.
Nov 27–Dec 3	Larger suite (more seeds/templates, timeout, shared-state and source-level faults, second model); stretch: recovery after other agents changed shared state post-fault.
Dec 4–10	Final experiments, figures, final report (due Dec 10).

B Revised Harness Validation

Setup. On October 29 we tested the revised harness on the UCSD-hosted `api-glm-5.3-flash` route through TritonAPI, with strict JSON output. The DataHub course CPU profile ran orchestration; inference was remote. The route is not an immutable checkpoint. Per-case results, configurations and package versions are in the repository; raw traces and responses are archived separately.

Live pilot. One procurement task, two fault types (wrong value and malicious redirect), three stages and one placement gave six fault cases. Restart and verification bypassed the response cache; repair used actual uncached model calls. The first clean run took 16.3 s and the uncached repeats 13.3 s and 6.0 s; median restart and verification times were 7.53 s and 17.32 s. These include network and local instrumentation, and are this route's only live timings without request spacing. The clean run passed, but only one of two uncached clean repeats passed. Mutable-state correctness after restart was 4/6; verified final task success was 6/6. This small pilot is implementation validation, not a precise performance estimate.

Propagation validation. Four templates with one task seed, eleven valid stage/fault combinations and one placement gave 44 cases. Clean baseline success was 3/4; eight uncached clean repeats passed 7/8. Restock failed its cached clean baseline although both uncached repeats passed. We retain all 44 cases and separately evaluate the 33 cases whose baseline completed cleanly:

Policy	Final success
No recovery	11/33
Full restart	29/33
Verify + actual retry	31/33

Always-verify detected the injected faults in all 44 cases but achieved final task success in only 40/44, with no attacker-directed actions remaining. Detection therefore does not guarantee correct repair. Restock inability is not counted as attack resistance. Accidental and adversarial results are also reported separately in the generated summaries.

Controls and failures. This second validation used cached replay and a 1.25 s minimum spacing for live requests. Its timings are not comparable to the live pilot as serving-performance measurements. A preceding four-worker batch failed on four requests before completing any fault case; its manifest and partial traces are retained, and it is excluded from success-rate denominators. Changed prompts still resample, and on this route the uncached clean repeats differed from the cached clean run in 3.0 units on average (Claude: 0.38), about as many as a researcher fault changes downstream (2.75). Its propagation figures therefore need repeated samples per case. No result here measures implemented selective recovery.

Resource accounting. There were 1,093 real requests across probes, pilot, the failed batch and serial validation. At the campus-listed rates of \$0.15/\$0.50 per million input/output tokens, completed usage cost is estimated at \$0.1761. Four failed requests retain \$0.0141 in conservative reservations, giving about \$0.1902 charged-or-reserved against a \$5 cap. These are token-rate estimates, not an authoritative billing statement. Credentials stayed in DataHub environment variables and are excluded from all exported artifacts.